



ITM Business
School
ITM SKILLS UNIVERSITY



ITM SKILLS UNIVERSITY

B.Tech Computer Science Engineering

JOB APPLICATION PORTAL

A Recruitment Platform using Node.js, Express.js & MongoDB

Submitted in Partial Fulfilment of the Requirements for the Course:

Backend Development – Node.js, Express.js & MongoDB

1. Students Details

Submitted By:

Name	Student ID
Tejas Chavan	150096725185

Programme: B.Tech Computer Science Engineering

Institution: ITM Skills University

Academic Year: 2025–2029

Contents

1. Students Details	1
2. Problem Statement and Project Objectives	2
3. System Architecture and Workflow	3
4. Database Design	5
5. Backend Implementation	7
6. Execution Screenshots	10
7. Results and Observations	13
8. Conclusion and Future Scope	14

2. Problem Statement and Project Objectives

2.1 Problem Statement

A recruitment platform allows candidates to browse job postings, upload resumes and apply for jobs. Recruiters can create job postings and review the applications that they receive for them.

The system must ensure that:

- A candidate **cannot apply to the same job more than once**.
- Only the recruiter who created a job can **view its applicants**.
- Resumes are **uploaded and stored** correctly.
- **Authentication** is required for protected operations.

2.2 Why This Project?

Recruitment handled through e-mail or spreadsheets is slow and error-prone. Duplicate applications create noise, resumes get misplaced, and applicant details can reach people who should not see them. A backend with strict rules enforced on the server solves these problems at the source rather than relying on users to behave correctly.

Proposed Solution

A REST API built with Node.js and Express.js stores users, jobs and applications in MongoDB through Mongoose. JWT protects restricted routes, Multer stores resume files, and two server-side checks (a duplicate check and an ownership check) guard the critical operations, as shown in Figure 1.



Figure 1: Proposed solution overview

2.3 Objectives

No.	Objective	Description
1	Candidate & Recruiter Authentication	Separate registration and login for both user types.
2	Job Management	Recruiters create, update, delete and list job postings.
3	Resume Upload using Multer	Candidates attach a resume file when applying.
4	Duplicate Application Prevention	One application per candidate per job.
5	JWT-based Authorization	Token-based access control on protected routes.
6	Recruiter Ownership Verification	Applicants are visible only to the job's creator.

3. System Architecture and Workflow

3.1 System Workflow

The platform serves two kinds of users. Figure 2 shows the candidate journey, from registration to a stored application, including the duplicate check. Figure 3 shows the recruiter journey, including the ownership verification that guards the applicant list.

Candidate workflow

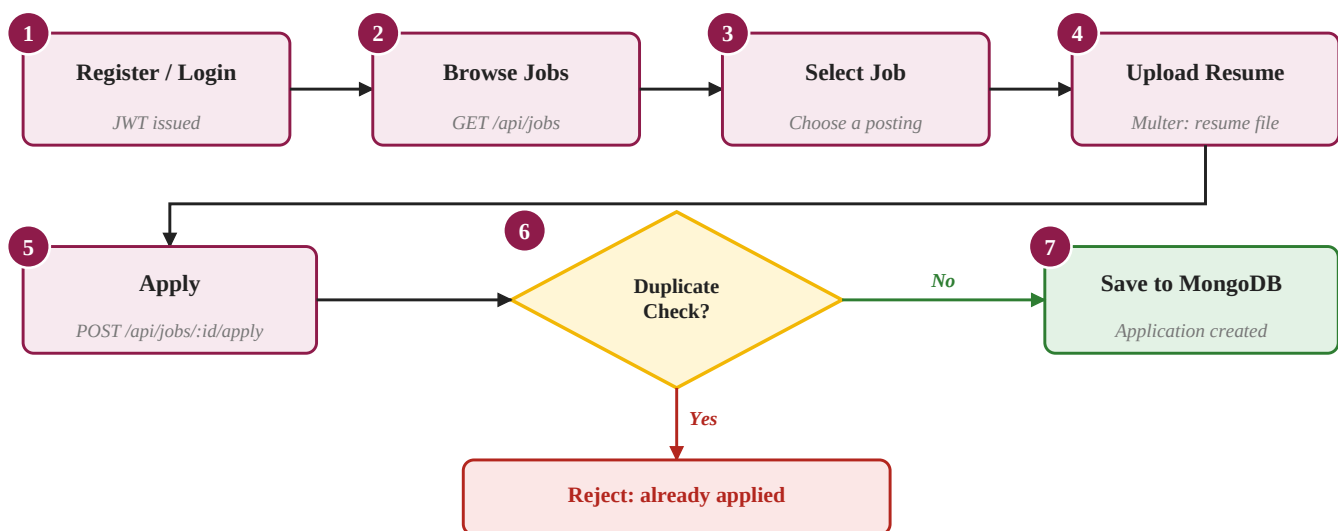


Figure 2: Candidate workflow

Recruiter workflow

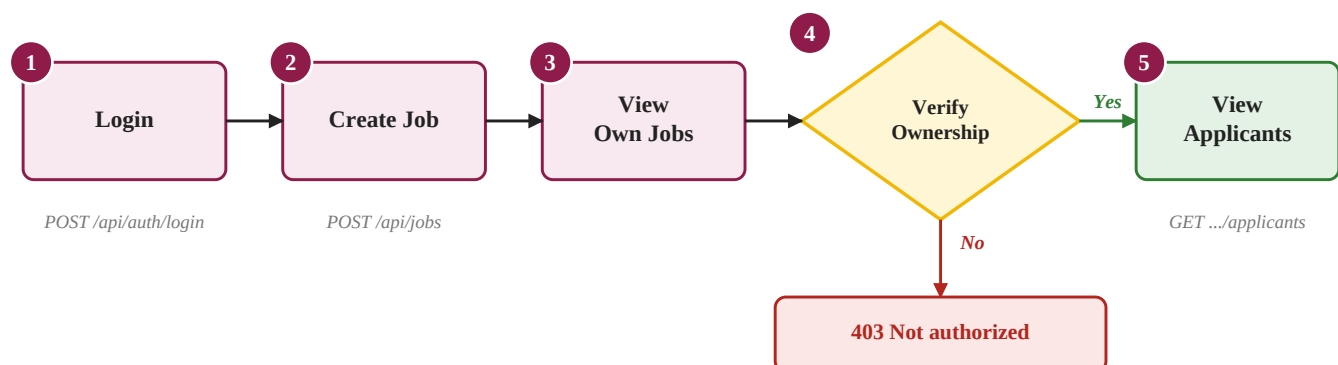


Figure 3: Recruiter workflow

Step	Stage	Description
1	Authentication	Candidate or recruiter registers and logs in; the API returns a JWT.
2	Job Listing	Candidates browse jobs; recruiters create and manage their own postings.
3	Resume Upload	Multer receives the resume file and stores it on the server.
4	Application	The candidate submits an application for the selected job.
5	Duplicate Check	The API rejects the request if the candidate already applied to that job.
6	Ownership Check	Before listing applicants, the API confirms the recruiter owns the job.
7	Storage	Users, jobs and applications are saved in MongoDB through Mongoose.

3.2 System Architecture

The architecture follows a layered design (Figure 4). The React frontend talks to the Express.js API over REST. The API applies authentication (JWT) and file handling (Multer), and uses Mongoose to read and write MongoDB Atlas. Resume files are stored in an uploads folder and referenced from the application document by path.

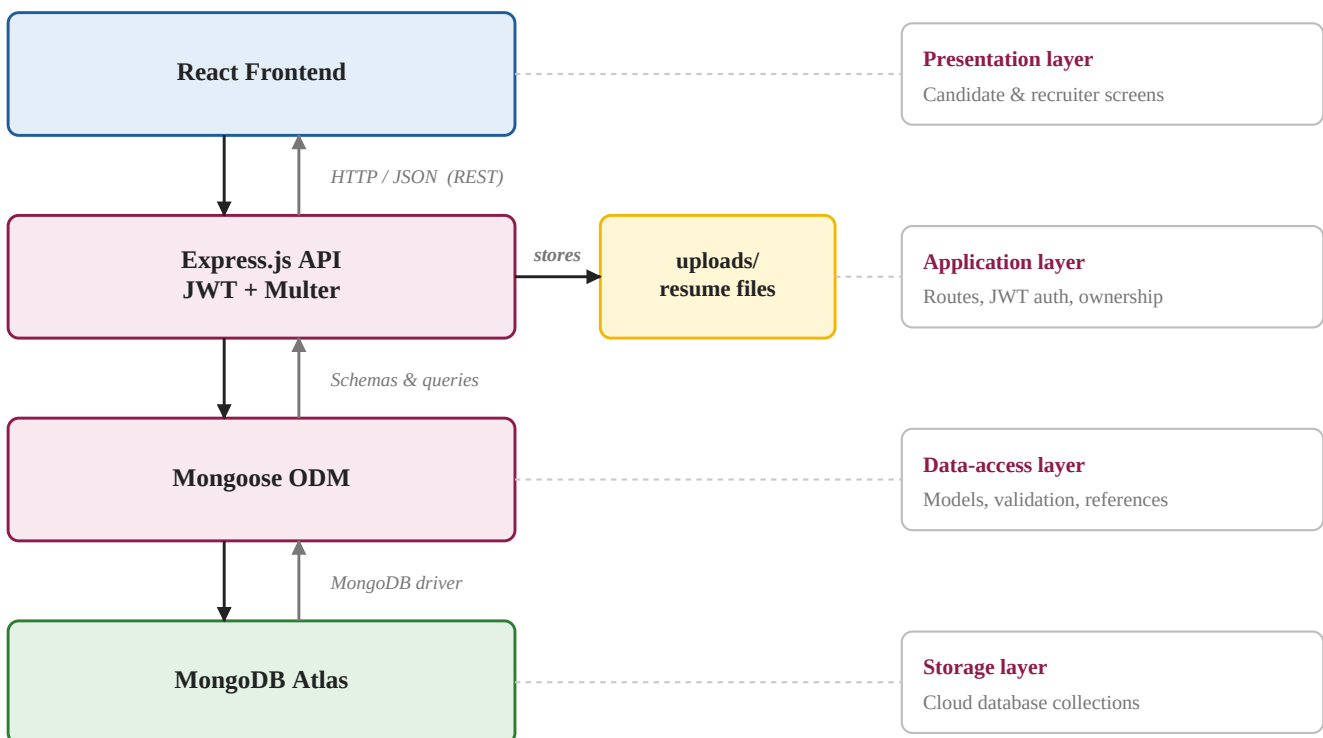


Figure 4: System architecture

3.3 Technology Stack

Layer	Technology	Role
Frontend	React	User interface for candidates and recruiters
Backend	Node.js, Express.js	REST API and business rules
Authentication	JSON Web Token (JWT)	Stateless login and protected routes
File Upload	Multer	Handles multipart resume uploads
ODM	Mongoose	Schemas, references and queries
Database	MongoDB Atlas	Cloud document database
API Testing	Postman	Manual verification of endpoints

4. Database Design

4.1 Main Entities

Entity	Purpose
Candidate	Stores candidate information
Recruiter	Stores recruiter information
Job	Stores job postings
Application	Stores candidate applications

4.2 Relationships

Relationship	Cardinality	Explanation
Recruiter → Job	1 : M	One recruiter can create many jobs.
Candidate → Application	1 : M	One candidate can submit many applications.
Job → Application	1 : M	One job can receive many applications.

4.3 ER Diagram

Figure 5 shows the entities with their attributes. The Application collection links a candidate to a job, so the pair (candidate, job) is exactly what the duplicate check inspects.

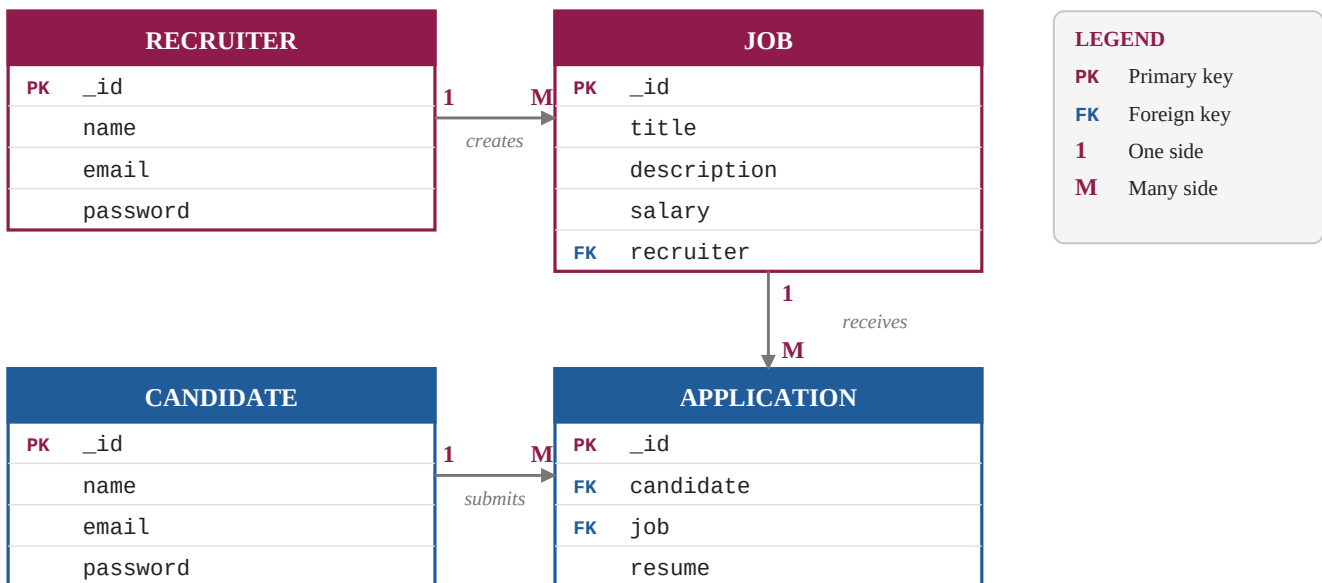


Figure 5: Entity-relationship diagram of the Job Application Portal

4.4 Schema Structure

Collection	Fields	References
Candidate	name, email, password	—
Recruiter	name, email, password	—
Job	title, description, salary, recruiter	recruiter → Recruiter ID
Application	candidate, job, resume	candidate → Candidate ID; job → Job ID; resume = file path

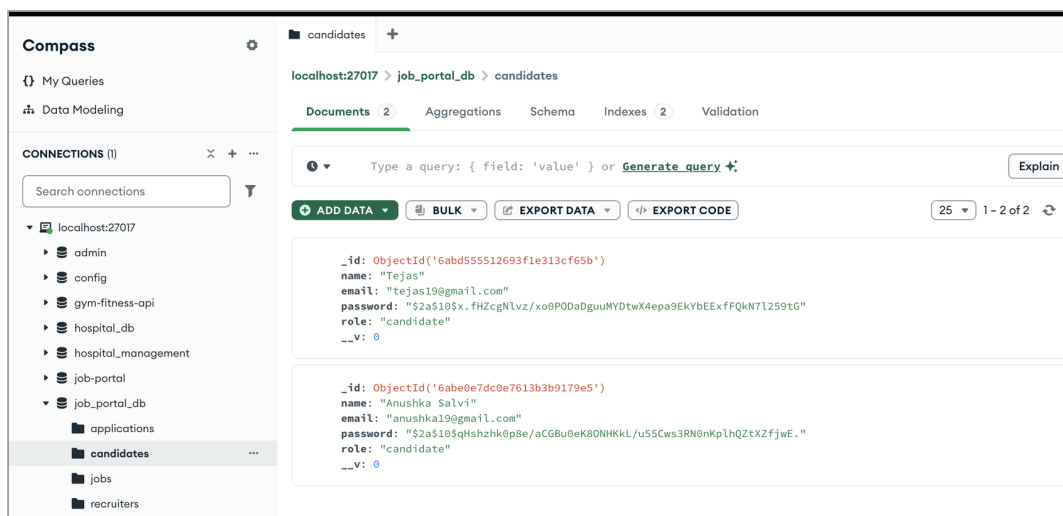


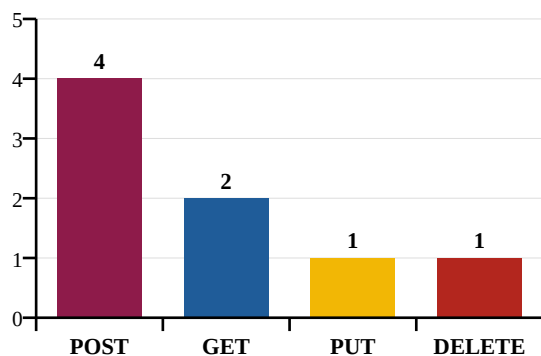
Figure 6: MongoDB Database / Collection Screenshot

5. Backend Implementation

5.1 REST APIs

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register	Public
POST	/api/auth/login	Login	Public
GET	/api/jobs	View jobs	Logged-in user
POST	/api/jobs	Create job	Recruiter
PUT	/api/jobs/:id	Update job	Recruiter (owner)
DELETE	/api/jobs/:id	Delete job	Recruiter (owner)
POST	/api/jobs/:id/apply	Apply	Candidate
GET	/api/jobs/:id/applicants	View applicants	Recruiter (owner)

Endpoints by HTTP method



Endpoints by module

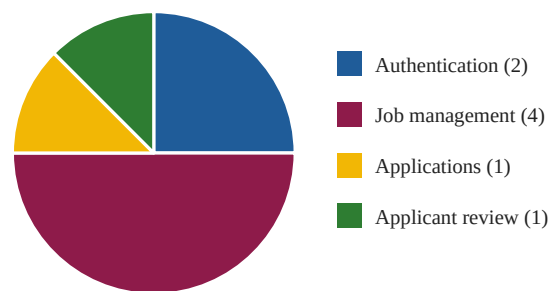


Figure 7: Distribution of the eight REST endpoints by HTTP method and by module

5.2 Authentication

JSON Web Tokens are used for authentication. After a successful login the server signs a token containing the user's id. The client sends it with every protected request, and a middleware verifies it and exposes the id as `req.user.id`.

```
const token = jwt.sign(
  { id: user._id },
  process.env.JWT_SECRET
);
```

5.3 Resume Upload

Multer is used to handle resume uploads. The `upload.single("resume")` middleware runs before the controller, stores the file on the server and exposes its details to the handler, which saves the file path in the Application document.

```
router.post(
  "/apply",
  upload.single("resume"),
  applyJob
);
```

5.4 Duplicate Application Check

Before creating an application, the controller looks for an existing document with the same candidate and job. If one exists, the API returns an error and nothing is saved. Figure 8 shows the complete apply pipeline.

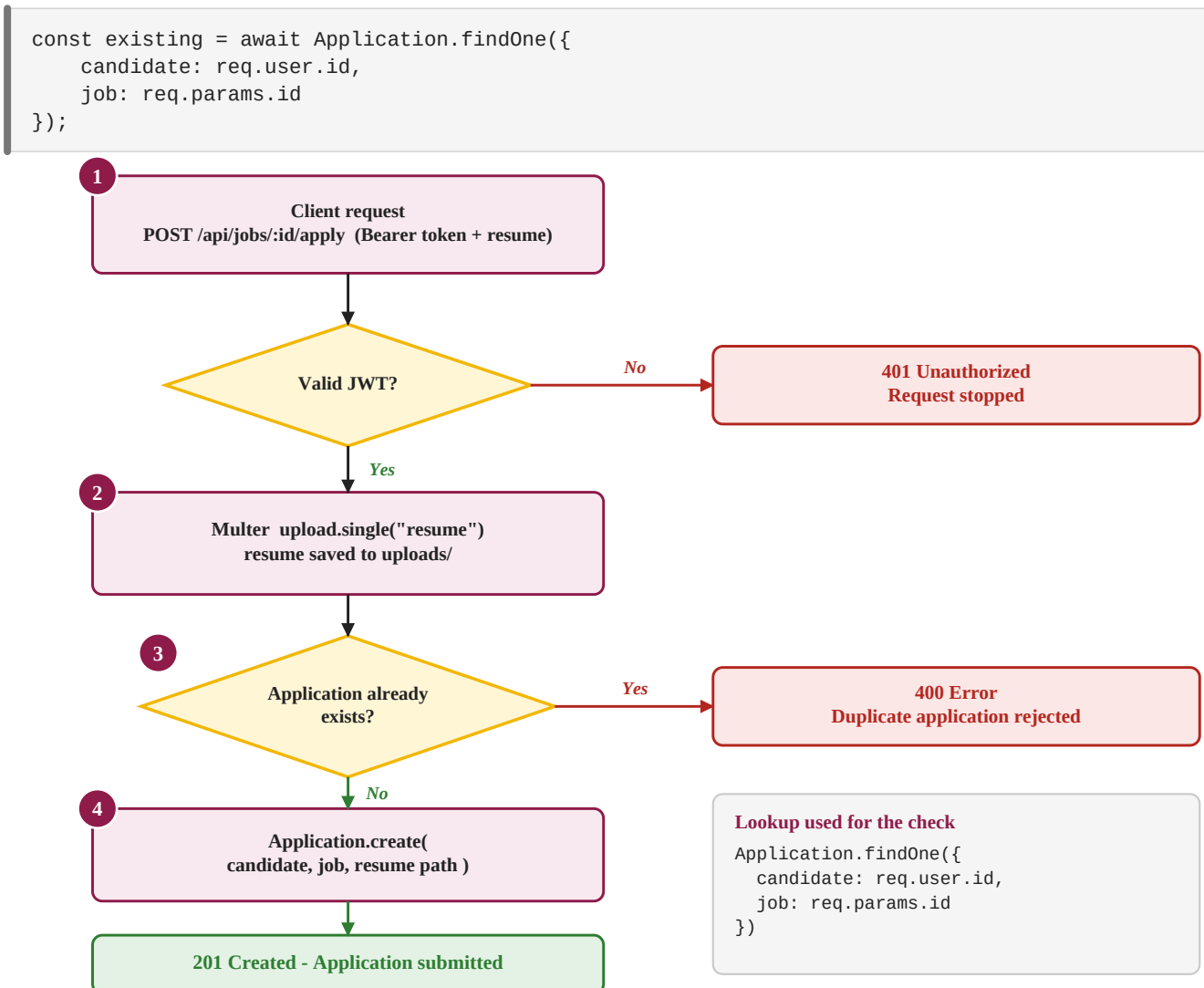


Figure 8: Apply-to-job request pipeline with JWT, Multer and duplicate check

Suggestion: Because the check and the insert are two separate database operations, two simultaneous requests could in theory both pass the check. A unique compound index on (candidate, job) in the Application schema would make the database itself reject the second insert. This can be added as a hardening step.

5.5 Ownership Authorization

Only the recruiter who created a job may see its applicants. The controller compares the job's recruiter id with the id from the verified token and answers with HTTP 403 when they differ (Figure 9).

```
if (job.recruiter.toString() !== req.user.id) {
  return res.status(403).json({
    message: "Not authorized"
  });
}
```

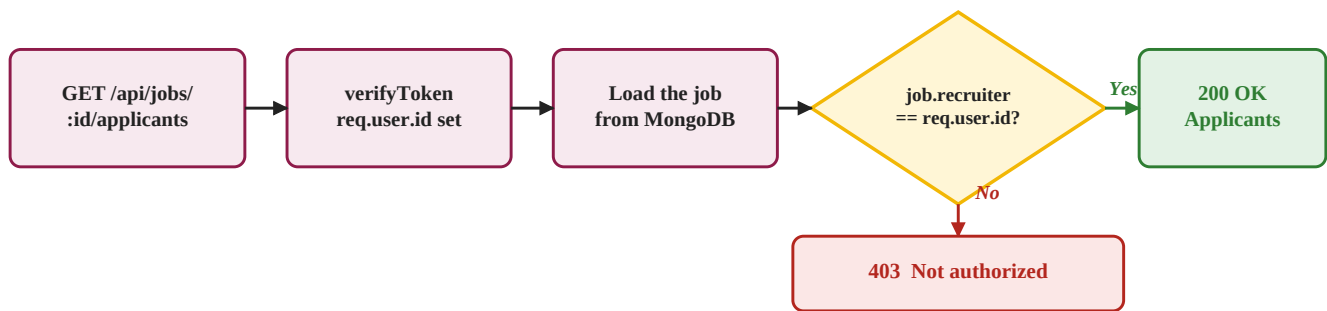


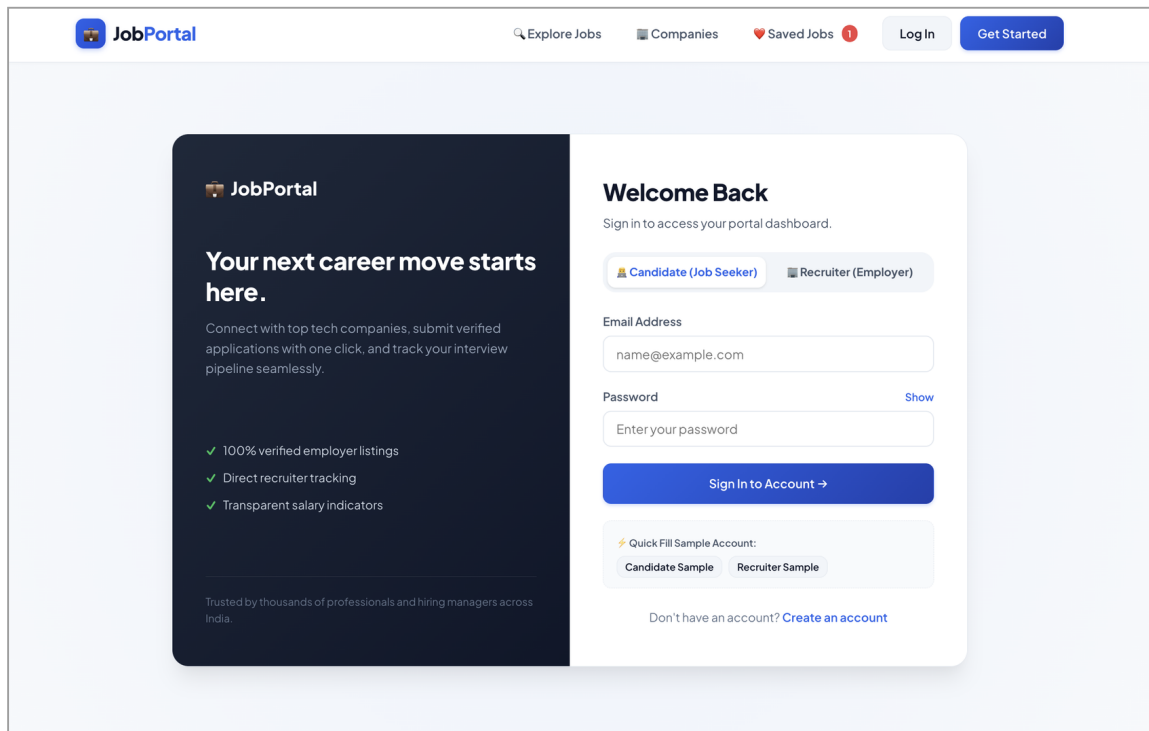
Figure 9: Recruiter ownership verification before listing applicants

5.6 Requirements and How They Are Enforced

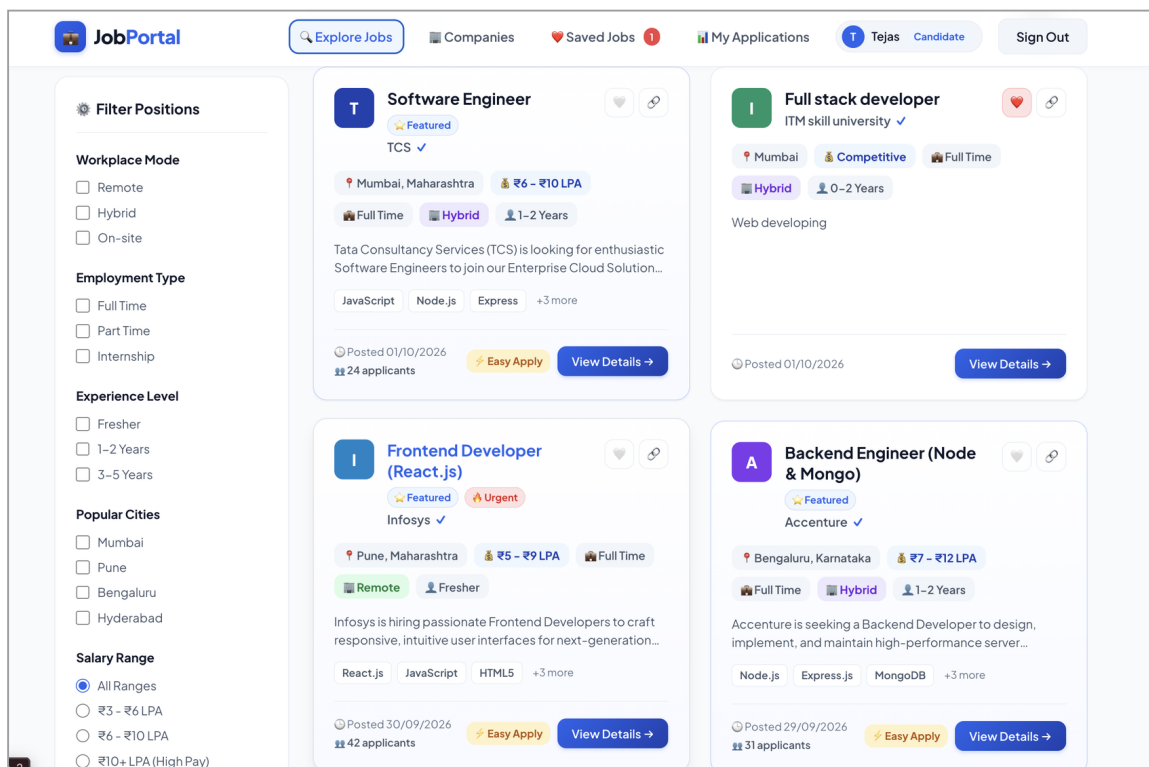
Requirement	Mechanism
No duplicate applications	Application.findOne({ candidate, job }) before insert
Only the owner sees applicants	job.recruiter compared with req.user.id; 403 on mismatch
Resumes stored correctly	Multer upload.single("resume"); file path saved in Application
Authentication for protected operations	JWT signed at login and verified on protected routes

6. Execution Screenshots

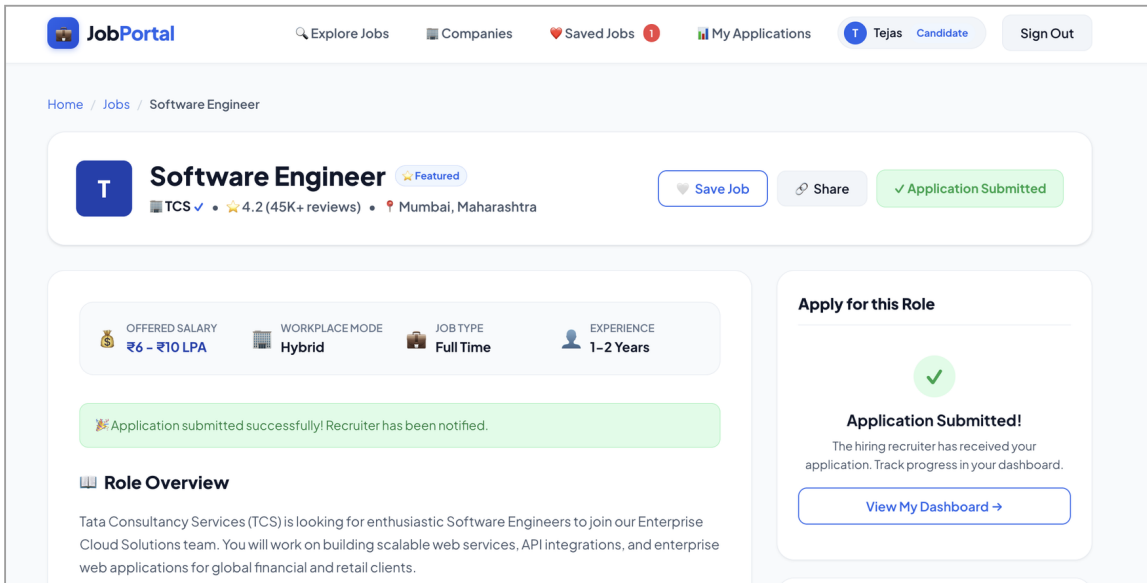
This section presents the execution of the application and its key operations: authentication, job listing, applying with a resume, the recruiter dashboard, the running backend server and the MongoDB collections.



Screenshot 1: Login / Register



Screenshot 2: Job Listing



JobPortal Explore Jobs Companies Saved Jobs 1 My Applications Tejas Candidate Sign Out

Home / Jobs / Software Engineer

Software Engineer Featured Save Job Share ✓ Application Submitted

TCS ✓ 4.2 (45K+ reviews) • Mumbai, Maharashtra

OFFERED SALARY ₹6 - ₹10 LPA WORKPLACE MODE Hybrid JOB TYPE Full Time EXPERIENCE 1-2 Years

Application submitted successfully! Recruiter has been notified.

Role Overview

Tata Consultancy Services (TCS) is looking for enthusiastic Software Engineers to join our Enterprise Cloud Solutions team. You will work on building scalable web services, API integrations, and enterprise web applications for global financial and retail clients.

Apply for this Role

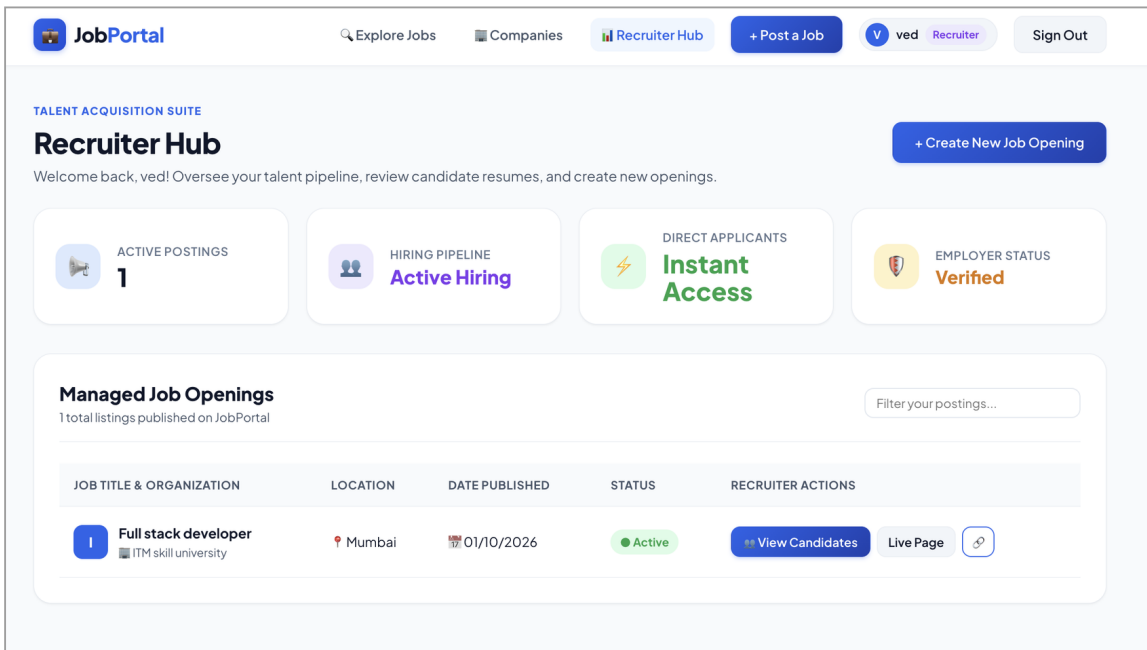
✓

Application Submitted!

The hiring recruiter has received your application. Track progress in your dashboard.

[View My Dashboard →](#)

Screenshot 3: Apply with Resume



JobPortal Explore Jobs Companies Recruiter Hub + Post a Job ved Recruiter Sign Out

TALENT ACQUISITION SUITE

Recruiter Hub + Create New Job Opening

Welcome back, ved! Oversee your talent pipeline, review candidate resumes, and create new openings.

ACTIVE POSTINGS 1

HIRING PIPELINE **Active Hiring**

DIRECT APPLICANTS **Instant Access**

EMPLOYER STATUS **Verified**

Managed Job Openings Filter your postings...

1 total listings published on JobPortal

JOB TITLE & ORGANIZATION	LOCATION	DATE PUBLISHED	STATUS	RECRUITER ACTIONS
Full stack developer ITM skill university	Mumbai	01/10/2026	Active	View Candidates Live Page Share

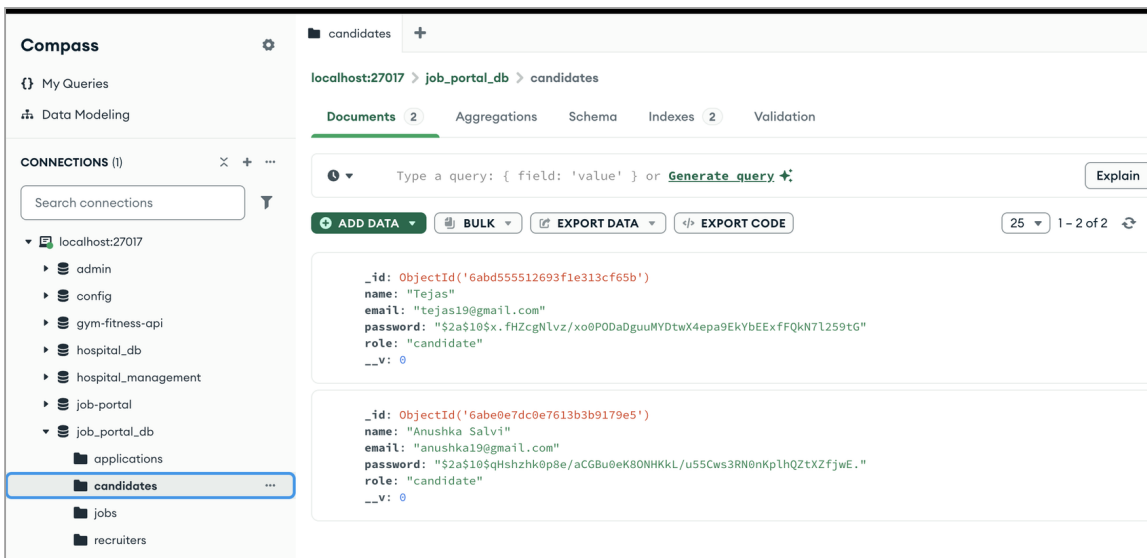
Screenshot 4: Recruiter Dashboard

- tejaschavan1907@TEJASs-MacBook-Air-3 BACKEND SEM3 % cd backend
- tejaschavan1907@TEJASs-MacBook-Air-3 backend % npm run dev

```
> job-portal-backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on port 5001
MongoDB connected
█
```

Screenshot 5: Backend Server Running (npm run dev, MongoDB connected)



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' sidebar lists various databases, with 'job_portal_db' selected. The main panel displays the 'candidates' collection in 'job_portal_db' at 'localhost:27017'. The 'Documents' tab is active, showing two documents in a list view. Each document contains fields for '_id', 'name', 'email', 'password', and 'role'.

Document 1	Document 2
<pre>{ "_id": ObjectId('6abd55512693f1e313cf65b'), "name": "Tejas", "email": "tejas19@gmail.com", "password": "\$2a\$10\$xx.fhZcgNlvz/xo0PODaDguuMYDtwX4epa9EkYbEEExfFQkN7L259tG", "role": "candidate", "__v": 0 }</pre>	<pre>{ "_id": ObjectId('6abe0e7dc0e7613b3b9179e5'), "name": "Anushka Salvi", "email": "anushka19@gmail.com", "password": "\$2a\$10\$qHshzhk0p8e/aCGBu0eK8ONHKKL/u55Cws3RN0nKp1hQZtXZfjwE.", "role": "candidate", "__v": 0 }</pre>

Screenshot 6: MongoDB Collections (job_portal_db)

7. Results and Observations

7.1 Results

Feature	Result	Feature	Result
Candidate Registration	Successful	Job Application	Successful
Recruiter Registration	Successful	Duplicate Application Prevention	Successful
Job Creation	Successful	Recruiter Authorization	Successful
Job Listing	Successful	Applicant Viewing	Successful
Resume Upload	Successful	MongoDB Storage	Successful

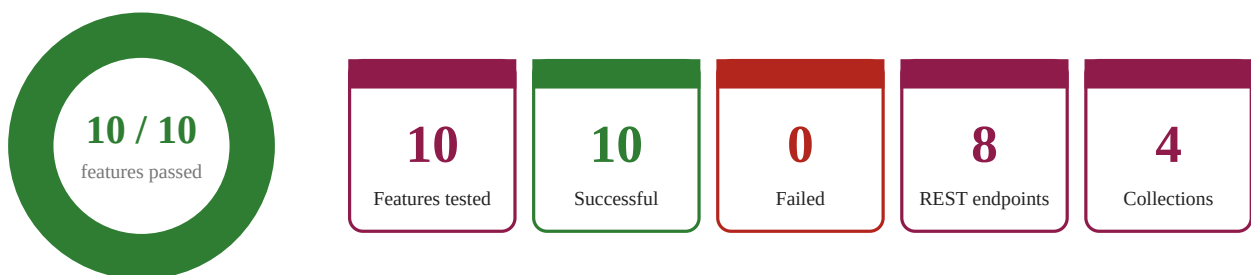


Figure 10: Summary of test outcomes

7.2 Key Findings

1. JWT successfully protects restricted routes.
2. Multer handles resume uploads.
3. Candidate and job references are stored using MongoDB/Mongoose.
4. Duplicate applications are rejected.
5. Recruiters can only access applicants belonging to their own jobs.

8. Conclusion and Future Scope

8.1 Conclusion

The Job Application Portal demonstrates a complete backend recruitment workflow using Node.js, Express.js and MongoDB. The system implements authentication, REST APIs, resume uploads, duplicate application prevention and ownership-based authorization.

The project also shows how the different parts of a backend cooperate to enforce business rules:

Concept	Learning	Concept	Learning
Express.js	Routing and REST API design	Multer	Handling file uploads
JWT	Stateless authentication	Mongoose	Schemas and references
Middleware	Reusable request checks	Authorization	Ownership-based access control

8.2 Future Scope

The system can be extended along the path shown in Figure 11. The principal enhancements envisaged are listed below.

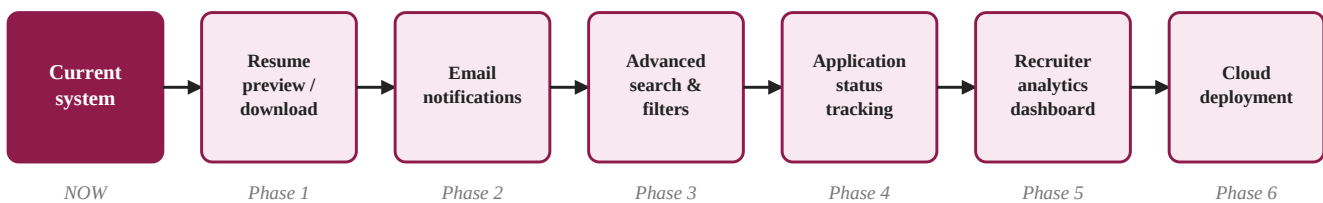


Figure 11: Future enhancement roadmap

- Online resume preview/download
- Email notifications
- Advanced job search and filtering
- Recruiter analytics dashboard
- Application status tracking
- Cloud deployment